Setting Up Full Multigrid

Louis Moresi [1]

[1] Australian National University

Keywords Underworld Code, Tricks of the Trade, development

Most of the effort in a geodynamics model is spent on solving the Stokes equations, and most of that goes into the velocity solver. How that is preconditioned Determines how long a model takes to run, and it is one of the few places where the right choice of settings can be worth an order of magnitude speed-up.

1. What multigrid does

An iterative solver reduces error unevenly. Simple relaxation methods (smoothers) are good at removing error that varies rapidly from cell to cell, and poor at removing error that varies smoothly across the whole domain. The smooth part then becomes the expensive part: on a fine mesh it takes many sweeps to move information from one side of the model to the other, and many sweeps to eliminate long-wavelength errors.

Multigrid takes that information and turns it on its head: error that is considered smooth on a fine mesh is *not* so smooth on a mesh twice as coarse, where it spans half as many cells, so a few smoothing sweeps on the coarse mesh can remove errors the fine mesh could not.

The multigrid method is a recursion: smooth on the fine mesh, transfer what is left to a coarser one, solve there (with another recursive step), transfer the correction back, smooth again. Each *level* handles the band of error it can see cheaply, and the work per unknown stops growing as the model gets bigger.

Two things have to be supplied: the coarse levels, and the operators that move between them.

2. Two ways to build the coarse levels

Algebraic multigrid (AMG) builds different resolutions of the problem directly from the matrix. It inspects the operator's connectivity, groups unknowns that are strongly coupled, and constructs coarse levels and transfer operators from that grouping alone. It needs very little from the mesh, and it works everywhere and is the sensible default. PETSc's implementation is GAMG, and it is what Underworld uses when it has nothing better.

Geometric multigrid (GMG) uses actual coarser meshes. If the fine mesh was built by ***refining*** a coarse one, those coarser meshes already exist, and the transfers follow from the refinement relation: each fine node either coincides with a coarse node or lies inside a known coarse cell. In PETSc this is `pc_type=mg`, and run as a *full* multigrid cycle — starting by solving entirely on the coarse level on the coarsest level and working to finer and

PUBLISHED Aug 14, 2026 ARTICLE UWTN 2026-014 VERSION 1.0.0 LICENCE CC-BY-4.0 UNDERWORLD underworld3 development @ 0addec15 SOURCE https://www.underworldcode.org/setting-up-full-multigrid/

finer renditions of the solution (cycling up and down through the meshes as it goes to accelerate removing the errors). We'll refer to this as FMG.

The difference matters when the operator's connectivity is a poor guide to the geometry, which happens when there are jumps or steep gradients in material properties. The algebraic method sees a matrix whose couplings are dominated by the stiff region and groups unknowns accordingly; the geometric method just uses the grids it has been given. It's probably not obvious which of these is a better choice so let's take a look and see. But first we'll need to see how to switch on the geometrical, full-multigrid solvers in Underworld and that starts with a multi-resolution mesh.

3. Making a mesh with a hierarchy

A mesh only carries a hierarchy if it was built with one. That is the `refinement` argument:

```
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)
len(mesh.dm_hierarchy)     # 3: the base and two refinements
```

The mesh you get back is the finest level, and the coarser ones are stored within it as PETSc `DMPlex` objects in `mesh.dm_hierarchy`. They are what the preconditioner uses. They are not Underworld meshes: there is no `Mesh` object for a coarse level, and extracting one takes work: so the hierarchy is something the solver reads rather than something you need to worry about.

Build the same mesh at `cellSize=0.0125` with no refinement and you get about the same resolution with no hierarchy at all — and no geometric multigrid. The algebraic multigrid works fine on a mesh that has a hierarchy, but it does not use the coarse grid information.

The solver picks it up on its own:

```
stokes = uw.systems.Stokes(mesh)
stokes.solve()                      # preconditioner defaults to "auto"
```

`"auto"` uses geometric multigrid when `len(mesh.dm_hierarchy) > 1` and GAMG when it does not. You can also ask directly, with `"fmg"` or `"gamg"`. Two behaviours are deliberate: `"auto"` only ever adds geometric multigrid to an untouched configuration, so it will not overwrite preconditioner options you have set yourself; and where a request cannot be honoured, you can find the reasoning in `stokes.pc_fallbacks`. `stokes.preconditioner_settings` reports what was actually applied.

4. Curved boundaries need help to refine.

Refining a mesh puts new nodes at the midpoints of existing edges. On a straight boundary that is fine. On a curved one it is not: the coarse mesh approximates a circle by a polygon, and the midpoint of a chord lies inside the circle, not on it. Those mid points need to be moved to lie on the boundary each time the mesh is refined.

Underworld's curved meshes carry a refinement *callback* that snaps boundary-labelled nodes back onto the true surface after each refinement. For the annulus it is a few lines — take the nodes labelled `Upper` and `Lower` and rescale each to the correct radius:

```
coords[upper] *= radiusOuter / R[upper]
coords[lower] *= radiusInner / R[lower]
```

This happens automatically for the built-in meshes, so in normal use there is nothing to do. It matters if you build a mesh of your own with curved boundaries: without a callback of this kind the mesh has facets that match the coarsest mesh and do not represent the boundary at the finest resolution.

5. What the coarse mesh leaves behind

Because the fine mesh is made by subdividing the coarse one, the coarse mesh's own triangulation is still visible in it. Its edges survive as continuous lines across the fine mesh, and its vertices remain places where an unusual number of elements meet. The pattern is a record of how the mesh was made rather than of the problem being solved.

`mesh.relax()` moves the nodes to improve element shapes while keeping the resolution and the topology, which loosens that imprint.

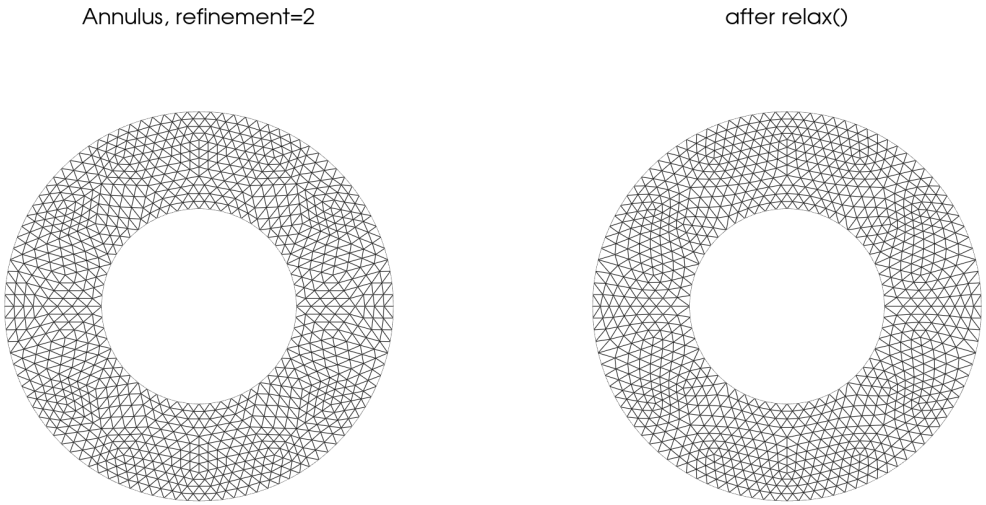

Figure 1: A twice-refined annulus, before and after `relax()`. The seams on the left are the coarse mesh's own edges, still visible after two refinements. Median element quality goes from 0.940 to 0.972 and the tenth percentile from 0.839 to 0.927; the worst single cell goes the other way, 0.827 to 0.763, because the mover minimises a global energy and will trade a few cells for many. Every node that began on a bounding circle is still exactly on it: relaxation moves interior coordinates only.

Because relaxation moves coordinates but not topology, the hierarchy survives which is the subject of a related technical note.

6. When the choice matters

The test is `SolKz`, a standard Stokes benchmark on the unit box: viscosity varying exponentially with depth, $\eta = e^{2Bz}$, forced by $\boldsymbol{f} = (0, \sin(m\pi z)\cos(n\pi x))$ with $n = 3$, $m = 2$, free slip on all four walls, Taylor–Hood P_2–P_1 elements. The viscosity contrast across the box is e^{2B}.

Effort is reported per unknown and relative to the cheapest FMG run - a ratio that we expect holds up reasonably well when we move from one platform to another. Each number is the median of three runs, and the worst run-to-run spread was 5%.

First, cost against viscosity contrast at a fixed resolution of 11 727 unknowns:

preconditioner	viscosity contrast	relative work per unknown	relative time per unknown
FMG	10^0	1.0	1.0
FMG	10^2	1.3	1.1
FMG	10^4	1.6	1.2
FMG	10^6	3.8	2.7
GAMG	10^0	9.1	3.5
GAMG	10^2	11.7	4.3
GAMG	10^4	13.1	4.8
GAMG	10^6	28.3	10.4

FMG costs about three times more at 10^6 than at constant viscosity; GAMG costs ten times more than the cheapest FMG run at constant viscosity, and ten times *that* by 10^6.

Second, cost against problem size at constant viscosity:

preconditioner	unknowns	relative work per unknown	relative time per unknown	Gflop/s
FMG	2 947	1.0	1.0	2.2
FMG	11 727	1.9	1.3	3.3
FMG	46 783	2.6	1.6	3.7
FMG	186 879	2.9	1.8	3.6
FMG	747 007	3.0	2.2	3.1
GAMG	2 947	6.2	2.1	6.7
GAMG	11 727	17.3	4.6	8.5
GAMG	46 783	55.3	13.6	9.1
GAMG	186 879	178.3	45.4	8.8

This is what multigrid exists for, and it needs the last row to see it. FMG's work per unknown climbs steeply at first and then stops: 1.0, 1.9, 2.6, 2.9, 3.0. Step by step the flop count goes as $N^{1.46}$, $N^{1.23}$, $N^{1.08}$, $N^{1.03}$ — converging on linear. That is the promise of multigrid, and it is only visible once the problem is large enough that the fixed costs stop dominating. GAMG goes the other way and settles: $N^{1.74}$, $N^{1.84}$, $N^{1.85}$.

The time column tells a different story, and the third column is why. FMG's wall clock sits at $N^{1.14}$ throughout, worse than its flop count, and the reason is in its rate: 2.2, 3.3, 3.7, 3.6, 3.1 Gflop/s — rising, then *falling* as the problem outgrows cache. GAMG holds 8.5–9.1 Gflop/s at every size, two and a half times FMG's.

This is the classic dilemma of an efficient algorithm. Geometric multigrid on an unstructured hierarchy does far less arithmetic, but it does it in a shape the processor dislikes: sparse, indirect, pointer-chasing, with small

coarse levels that cannot fill a pipeline. Algebraic multigrid does much more arithmetic in denser, more regular operators that run close to the machine's peak. The better algorithm cannot get near the hardware's sweet spot, and a good part of its advantage is handed back at the door — FMG does 61 times less work than GAMG at the largest size compared here, and is 24 times faster.

Which measure you want depends on the question. Flops per unknown is the property of the *method*, and it is deterministic and machine-independent — it is what tells you FMG is asymptotically $O(N)$ and GAMG is not. Wall clock is what you wait for on this machine today. Neither is the honest number on its own.

Underneath the timings there is a cleaner number. Solving the Stokes system does not call the velocity block once: the Schur factorisation invokes it sixteen times, and that count is the same for both preconditioners at every resolution, so it cancels from any comparison between them. What is left is how many multigrid cycles each of those velocity solves takes:

preconditioner	2 947 unknowns	11 727	46 783
FMG	3	3	3
GAMG	89–126	243–347	723–1129

FMG converges the velocity block in **three cycles, whatever the resolution**. That is the property multigrid is built to have, stated as plainly as it can be: the mesh grows sixteenfold and the work to solve does not move. Everything above linear in the timings is the cost of a cycle rising as the hierarchy deepens, not more cycles being needed. GAMG's count grows as roughly $N^{0.7}$–$N^{0.8}$, and that exponent is itself increasing.

One thing these counts still do not measure is cost. A geometric cycle and an algebraic one are different amounts of work, over different hierarchies with different smoothers and coarse solves, so three against a hundred is not a hundred-fold difference in effort — the timings above are what answers that. What the counts show is the shape: one flat, one growing.

> **Two things this comparison does not address**
>
> The solver tolerance is held fixed as the mesh refines. That is the usual way to show a multigrid scaling an does not account for the fact that a finer mesh has a smaller discretisation error, and would potentially require tightening the solver tolerance to reach that error.

Taken together the picture is of two solver choices that hold up well. Both converged on every problem here, from constant viscosity to a contrast of 10^6 and from three thousand unknowns to nearly two hundred thousand, without any tuning beyond the solver choice. GAMG converged more slowly, and with a worse scaling trend, but it is not fragile on this problem. We can, however, achieve better scaling and overall performance with the full-multigrid options and the set-up is quite straightforward.

7. Using it

```
# Build the hierarchy at mesh construction: base + two refinements.
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)

stokes = uw.systems.Stokes(mesh)
stokes.preconditioner = "auto"        # FMG when a hierarchy exists
```

```
stokes.solve()

print(len(mesh.dm_hierarchy))         # how many levels there are
print(stokes.preconditioner_settings) # what was applied
print(stokes.pc_fallbacks)            # anything declined, and why
```

If a solve is slow and the mesh has no hierarchy, this is the first thing to change: rebuild the mesh with `refinement` rather than at a fine `cellSize`, and the same resolution arrives with the coarse levels attached. Just be aware that the cell-size determines the coarsest mesh and is refined (by a factor of 2) each time.